

Roadmap Handoff Guide

Use this guide when you need to describe **milestones, execution-tier work items, delivery timing, dependencies, and success criteria over time**.

This guide is about the planned shape of delivery and the handoff from **strategic milestones** to **execution work**. It is not a task tracker in every technical detail, but it should be clear enough that a technical modeler or AI agent can transform it into a roadmap with both milestone and work-breakdown structure.

KNOWLEDGE AREA

This guide helps with **delivery milestones and work planning**. Technically, it covers **delivery planning and execution sequencing**.

Guide family map

Where this guide sits in the full Blueprint Handoff Atlas family.

ROOT & CROSS-CUTTING

Handoff Atlas

Blueprint Bundle

Metamodel Vocabulary

Migrations

DESIGN

Architecture

Concepts

Domain

Dynamics

Infrastructure

Interactions

Models

Quality

Rules

Story

GOVERNANCE

Capability

Decisions

Motivation

Organization

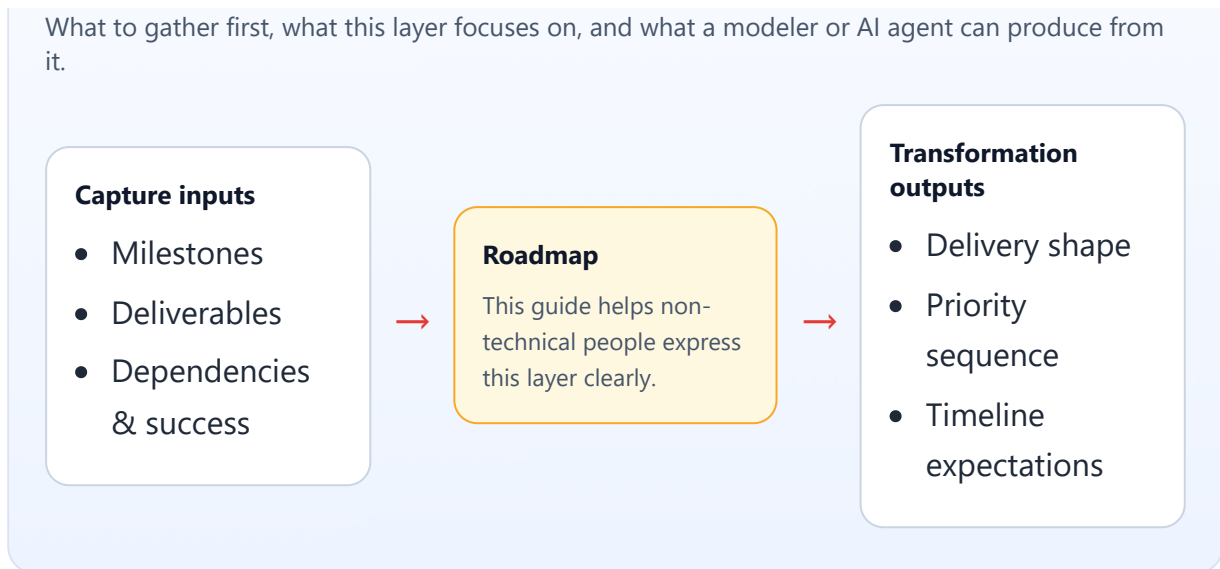
Roadmap

Test Cases

Value Stream

Capture → focus → transformation

What to gather first, what this layer focuses on, and what a modeler or AI agent can produce from it.



What belongs here

- milestones
- target dates or delivery windows
- optional sprint cadence anchors
- execution-tier work items such as epics, phases, foundations, subscopes, and tasks
- what ships in each milestone
- dependencies between milestones
- dependencies between work items or between work items and milestones
- blockers and sequencing constraints
- traceability from delivery work to goals, risks, decisions, value streams, stories, and use cases
- success criteria

What does not belong here

- low-level task tracking
- broad goals with no time or delivery framing
- technical implementation checklists
- renderer-specific styling or chart cosmetics

Core things to capture

- milestone name, purpose, target date, and status
- what is expected to be delivered in each milestone
- what success looks like for each milestone

- whether there is a sprint cadence anchor for planning by sprint number
- what execution work items exist beneath the milestone layer
- what kind of work item each one is (epic , phase , foundation , subscope , task)
- when work starts and ends, either by sprint or by explicit date
- who owns the milestone or work item
- what dependencies, blockers, or rollback-style delivery risks matter
- what goals, risks, decisions, value streams, stories, or use cases the work advances

Core relationships to capture

- milestone -> deliverables
- milestone -> dependency on another milestone
- milestone -> success criteria
- work item -> milestone it rolls up to
- work item -> work item dependency
- parent work item -> child work item
- work item -> blocker
- work item or milestone -> goals/risks/decisions/value streams/stories/use cases

Modus operandi

Use roadmap capture to show meaningful progress:

1. define milestone outcomes first
2. identify the execution work that makes each milestone real
3. decide whether planning is date-based, sprint-based, or mixed
4. group execution work into meaningful lanes such as epics, phases, or foundations
5. add nested subscopes or tasks only where they improve handoff clarity
6. state success criteria, dependencies, blockers, and ownership
7. connect roadmap items back to the goals, risks, decisions, value streams, stories, and use cases they serve

Think of this guide as a **delivery handoff** from intent to execution: a reader should understand what is being delivered, in what order, by whom, and why it matters.

Prompt set

- what is the milestone called?
- what value is delivered in it?

- when is it targeted?
- how will we know it is achieved?
- what must happen first?
- what work items make this milestone possible?
- are those work items better expressed by sprint or by date?
- what blocks them today?
- what goal, risk, decision, or value stream does each item support?

Free-text intake template

- **Milestone name:**
- **Milestone purpose:**
- **Target date/window:**
- **Milestone status:**
- **Deliverables:**
- **Success criteria:**
- **Milestone dependencies:**
- **Cadence anchor (if sprint-based planning is used):**
- **Execution work items:**
 - **Work item name:**
 - **Kind (epic/phase/foundation/subscope/task):**
 - **Description:**
 - **Milestone rolled up to:**
 - **Placement (start/end sprint or start/end date):**
 - **Confidence / progress:**
 - **Owner / executor:**
 - **Depends on:**
 - **Blockers:**
 - **Advances goals / mitigates risks / realizes decisions / supports value streams / delivers stories or use cases:**

Worked example

- **Milestone:** Employee Self-Service MVP
- **Purpose:** first usable employee leave workflow without manual HR entry
- **Target date:** 2026-10-01
- **Milestone status:** planned
- **Deliverables:** leave request submission, manager approval, dashboard summary
- **Success criteria:** pilot users can submit and receive leave decisions without manual HR entry

- **Cadence anchor:** sprint 30 starts on 2026-07-01 and lasts 14 days
- **Execution work items:**
 - **Epic:** Employee leave self-service
 - rolls up to MVP
 - start sprint: 30
 - end sprint: 35
 - advances goal: employee self-service
 - **Foundation:** approval policy integration
 - depends on policy rule clarification
 - blocker: unresolved compliance inquiry
 - **Task:** manager dashboard decision panel
 - executor: backend + frontend team
 - delivers related story and use case coverage

Layer-specific handoff guidance

For roadmap, the handoff should connect **strategy to execution** clearly.

The receiver should be able to see:

- which milestones matter and why
- what work items roll up to them
- how timing is expressed (date or sprint)
- what dependencies, blockers, and owners shape delivery

If roadmap notes mention milestones but not the work that realizes them, the handoff is incomplete.

How this becomes YAML

A technical modeler or AI agent can transform this into:

- milestones with target dates, deliverables, status, success criteria, and dependencies
- optional roadmap cadence for sprint-based planning
- execution work items with kind, timing, ownership, progress, blockers, nesting, and milestone roll-up
- typed traceability links back to goals, risks, decisions, value streams, stories, and use cases

Common mistakes

- naming a milestone without saying what is delivered
- omitting success criteria
- confusing milestones with execution work items
- forgetting dependencies or blockers
- planning work without saying whether timing is by sprint or by date
- treating roadmap items as isolated delivery pieces with no traceability back to strategy or stories

